

Evolution and State of the Art in Flow Control for Network-on-Chip Interconnects

With Emphasis on Ring and Hierarchical-Ring Topologies

Prepared by Ditto

June 15, 2026

Abstract

This report surveys the evolution of NoC flow control for multicore processors, from store-and-forward and virtual cut-through to wormhole, virtual-channel, credit-based, elastic-buffer, and bufferless/deflection designs. The emphasis is ring-based NoCs, including simple rings, hierarchical rings, and the HiRD design (“Design and Evaluation of Hierarchical Rings with Deflection Routing”) by Ausavarungnirun et al. The main conclusion is that rings remain attractive when layout regularity, low router cost, coherent traffic ordering, and moderate core counts dominate, but their flow-control choices are tightly coupled to injection fairness, ejection priority, and congestion isolation. For larger and less locality-friendly systems, the state of the art usually shifts toward meshes or hierarchical/high-radix designs using credit-based wormhole flow control, a small number of virtual channels, protocol-level virtual networks, and sometimes adaptive or bypass mechanisms. Bufferless/deflection flow control is the most ring-relevant research alternative because it converts backpressure into extra path length, but it must be engineered carefully to avoid livelock, starvation, and pathological latency.

1 Executive takeaways

1. **The industrial default is conservative.** Publicly documented NoCs in multicore processors and coherent fabrics usually emphasize lossless packet/flit transport, local backpressure or credits, separated traffic classes, and simple deadlock rules. This is less glamorous than some research proposals, but easier to time, verify, and integrate with coherence.
2. **Rings are not obsolete, but they are niche-optimal.** A ring has tiny routers, natural layout regularity, and simple arbitration. The cost is poor bisection bandwidth and distance growth. Hierarchical rings try to preserve ring simplicity locally while reducing global traversal distance.
3. **Ring flow control has three canonical families.** Slot/token/circuit-style rings avoid buffer competition by scheduling slots; credit or ready/valid rings apply backpressure when downstream buffers fill; deflection/bufferless rings avoid stopping by misrouting or re-circulating packets.
4. **HiRD’s core idea is to use deflection as the ring analogue of buffering.** Hierarchical Rings with Deflection Routing (HiRD) connects small local rings through a global ring and uses deflection at local/global interfaces and destinations to avoid expensive buffering and global backpressure [1]. This makes the router simple and buffer-light, but shifts the burden to fairness, livelock freedom, and injection/ejection policy.
5. **For non-ring NoCs, virtual channels remain the workhorse.** VC flow control, introduced by Dally [2, 3], is still the cleanest way to decouple traffic classes, reduce head-of-line blocking, provide escape resources for adaptive routing, and break coherence-protocol dependency cycles.

6. **The research frontier is conditional, not universal.** Express virtual channels, SMART, elastic buffers, and bufferless/minimally buffered designs each optimize a different bottleneck: hop latency, buffer area/leakage, or low-load latency. None dominates all workloads and physical design constraints.

2 What exactly is “flow control” in a NoC?

In a NoC, flow control decides when a sender may inject or forward a flit/packet into a downstream resource. It is distinct from routing, which chooses the path, and arbitration, which chooses among contenders for the same output. In practice the three are coupled: a router can only route through outputs whose downstream buffers, slots, or credits are available.

The central trade-off is simple: *buffer when blocked, stop the sender, reserve capacity ahead of time, or send the packet somewhere else*. Most NoC flow-control mechanisms are combinations of these four responses.

- **Store-and-forward:** buffer the whole packet at every hop. Simple, but too buffer-heavy and high-latency for many on-chip fabrics.
- **Virtual cut-through:** forward as soon as the next router can accept the whole packet; otherwise buffer the packet [4].
- **Wormhole:** split packets into flits; a header flit reserves route state and following flits pipeline through. This drastically reduces buffer capacity but creates channel-holding dependencies that can deadlock [5, 6].
- **Credit-based flow control:** upstream maintains a count of free downstream buffer slots. Sending consumes a credit; freeing a slot returns a credit. It is precise and lossless, but credit round-trip latency determines how much buffering is needed.
- **Virtual-channel flow control:** multiplex multiple logical queues over one physical link. VCs reduce head-of-line blocking and are the standard tool for deadlock avoidance, adaptive routing escape paths, and coherence traffic-class isolation [2, 3].
- **Elastic/bufferless/deflection:** reduce or eliminate input buffers. Elastic-buffer NoCs store flits in link pipeline registers [7]; bufferless deflection routes a contending flit to a non-minimal output rather than waiting [8, 9].

3 Historical evolution

3.1 Before NoCs: switching and deadlock foundations

Virtual cut-through, wormhole routing, and deadlock theory predate the NoC term. Dally and Seitz’s channel-dependency graph formalized when a routing function can deadlock: a cycle in the channel dependency graph is dangerous because packets can each hold one channel while waiting for the next [10]. Wormhole routing made this especially important because a blocked packet may hold buffers and links across multiple hops.

Dally’s virtual-channel work then introduced a practical hardware mechanism to split one physical channel into multiple logical channels [2, 3]. This matters because a physical wire can remain busy even when one logical flow is blocked. It also lets a designer impose acyclic resource ordering without duplicating physical links.

3.2 NoC era: packets, not global wires

The early-2000s NoC argument was that global buses and ad hoc wiring do not scale with core count and SoC complexity. Dally and Towles’ “route packets, not wires” paper and Benini/de Micheli’s NoC paradigm paper framed on-chip interconnects as packet-switched networks with explicit topology, routing, flow control, and router microarchitecture [11, 12].

The classic wormhole VC router pipeline has input buffering, route computation, VC allocation, switch allocation, switch traversal, and link traversal. Peh and Dally modeled these delays and proposed speculation to reduce pipeline cost [13]; Mullins et al. further optimized low-latency VC routers [14]. This is still the baseline mental model for a mesh NoC router.

3.3 Mid-2000s onward: reduce hop count or remove buffers

Two lines of work attacked NoC cost from opposite directions:

- **Bypass and high-radix designs** reduce the number or cost of router stops. Express virtual channels bypass intermediate routers [15]; flattened butterfly reduces diameter by using higher-radix routers [16]; SMART tries to traverse several routers in one cycle when setup arbitration succeeds [17].
- **Buffer reduction designs** reduce storage and allocator complexity. Elastic buffers place storage in links [7]. Bufferless and minimally buffered deflection routers remove most input buffering and route losers to alternate outputs [8, 9, 18, 19].

The practical lesson is that the best mechanism depends on whether the bottleneck is hop latency, buffer area/leakage, energy per bit, saturation throughput, or verification complexity. Modern industrial fabrics generally choose robust combinations rather than maximal academic cleverness.

4 Ring-based NoCs

4.1 Why rings are attractive

A ring router can be extremely small: one or two transit directions, injection/ejection ports, and local arbitration. Rings also provide a naturally pipelined layout around cores, last-level-cache slices, memory controllers, or coherent agents. That is why rings appeared in commercial designs such as Intel client/server processors and IBM Cell’s Element Interconnect Bus (EIB) [20, 21].

The downside is equally fundamental. A single unidirectional ring has bisection bandwidth proportional to link width, not node count. Mean distance grows with ring size. A bidirectional ring helps latency and bandwidth but adds direction choice, ordering concerns, and possible deadlock if packets can hold resources while turning or changing rings. Hierarchical rings reduce distance by using local rings for nearby traffic and a global ring for inter-cluster traffic [1, 22, 23].

4.2 Canonical ring flow-control styles

4.2.1 Slot, token, and circuit-style rings

In a slotted ring, each cycle or pipeline slot is either empty or carries a packet/flit. A node injects only when it sees an empty slot or owns a token/credit. IBM Cell’s EIB is a useful industry example: public analyses describe four data rings plus a command bus, pipelined circuit-switched transfers, and end-to-end command credits before elements issue command-bus requests [21]. Slot/token designs have excellent simplicity and bounded arbitration, but under bursty traffic unused slots and head-of-line effects can hurt utilization.

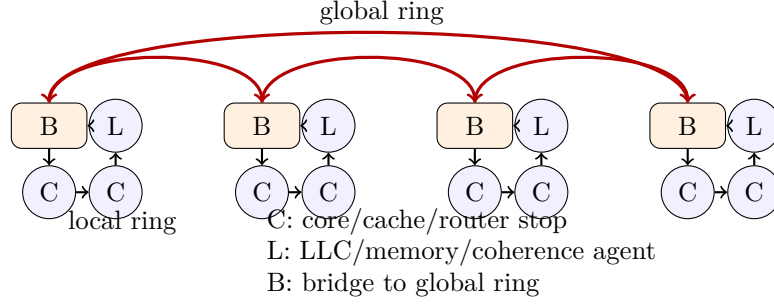


Figure 1: Simplified hierarchical-ring NoC. Local rings provide cheap nearby communication; bridge nodes transfer packets to a global ring. Deflection-based designs keep packets moving when ejection or bridge transfer is unavailable.

4.2.2 Credit/backpressure rings

A ring can also use local ready/valid or credit-based backpressure. If the next ring stop has no space, upstream stops forwarding. This is lossless and simple for short rings, but in a cycle it can create global stalls unless the design preserves empty buffers or orders resources carefully. Bubble flow control is the classic solution: allow injection only if enough empty buffer slots remain to guarantee at least one “bubble” circulates, breaking cyclic wait conditions [24]. In a wormhole ring or torus, bubble rules are a cheap alternative to adding many VCs, though they reduce usable buffering at high load.

4.2.3 Virtual-channel rings

VCS can break a ring’s cyclic dependency by splitting traffic into ordered classes or by using escape resources. This is common in torus and mesh theory and also relevant to rings that support bidirectional movement, multiple message classes, or coherence request/response interactions. The cost is that even a simple ring stop becomes a VC router with per-VC buffers, credits, and arbitration.

4.2.4 Deflection and bufferless rings

Deflection replaces “wait” with “go somewhere else.” In a ring, the simplest deflection is re-circulation: if a packet cannot eject or transfer to another ring, it continues around and tries later. In a hierarchical ring, a packet can also miss a bridge opportunity and keep circulating on the local or global ring. This avoids backpressure and input buffering but consumes extra bandwidth and can inflate tail latency.

The deflection family is attractive for rings because the output choice space is small and router critical paths can be kept short. However, correctness depends on four policies: ejection priority, injection throttling, age/fairness, and rules for transfer between local/global rings. Without those, a packet can be repeatedly bypassed by younger traffic, creating starvation or severe latency outliers.

5 Case study: Hierarchical Rings with Deflection Routing (HiRD)

Ausavarungnirun et al.’s HiRD paper is a useful focal point because it combines three ideas that are individually old but jointly compelling for manycore CMPs: hierarchy, rings, and deflection routing [1].

5.1 Topology motivation

A flat ring is elegant at small scale, but the average traversal distance and bisection bottleneck grow with node count. A mesh improves bisection and distance but uses more complex routers, multi-port allocators, and often VCs. A hierarchical ring tries to land between them:

- group nearby cores/cache slices on small local rings;
- connect those local rings through bridge routers to a global ring;
- keep local traffic local when possible;
- use the global ring only for inter-cluster traffic.

This is a topology-level answer to physical design: ring wires can be simple and predictable, local communication remains cheap, and global traffic no longer circles every node in the chip.

5.2 Flow-control mechanism

The flow-control problem in a hierarchical ring is concentrated at two places: ejection points and local/global bridge points. If a packet reaches its destination but the destination cannot accept it, a buffered design must stop or store it. If many packets want the bridge, a buffered design needs bridge queues and backpressure. HiRD’s philosophy is to avoid much of that storage by allowing packets to continue around the ring and try again later. In other words, the ring itself becomes a circulating holding structure.

Conceptually:

1. A packet on a local ring either ejects locally, transfers to a bridge/global ring if needed, or continues circulating.
2. A packet on the global ring transfers to the destination local ring when a bridge opportunity is available; otherwise it continues around the global ring.
3. Ejection and transfer must be prioritized over new injection often enough that in-flight packets make progress.
4. Injection control prevents newly injected packets from consuming all opportunities and starving deflected packets.

This is not “flow control disappears.” Rather, it moves from per-hop buffer credits to admission, ejection, and bridge-transfer policy. The design avoids global backpressure, but the designer still needs progress guarantees.

5.3 Why deflection fits rings particularly well

Deflection in a mesh can send packets onto non-minimal paths with many possible wrong turns. In a ring, the non-minimal path is often just another lap. That makes the mechanism simple: no complex output selection, and no large VC allocator. The cost is predictable too: extra laps consume ring bandwidth and increase latency variance.

For workloads with locality, local rings reduce pressure on the global ring, so deflection is relatively rare. For adversarial or all-to-all traffic, deflection becomes much more expensive because the global ring and bridge points become bottlenecks. This is the fundamental ring trade-off: cheap routers and wires, but hot bridges can dominate.

5.4 Comparison against bubble/credit rings

Bubble flow control and credit rings prevent overflow by stopping injection or forwarding. HiRD-style deflection prevents overflow by refusing to stop in-flight traffic. The distinction matters:

- **Credit/bubble:** lower path inflation, but stalls can propagate around cycles; requires buffer accounting and may reduce usable capacity to preserve bubbles.

- **Deflection:** simpler transit path and fewer buffers, but uses extra bandwidth under contention; requires fairness and injection throttling rather than downstream credits.
- **VC ring:** more robust for multiple traffic classes and strict deadlock isolation, but loses much of the tiny-router advantage that made rings appealing.

6 State of the art outside rings

Because ring-specific work is narrower than mesh/torus NoC work, a credible survey must include broader NoC flow control. The mechanisms below define the practical design space.

6.1 Credit-based wormhole with a few VCs

This is the workhorse design point. Packets are flitized; routers use small input buffers; credits provide lossless local backpressure; VCs separate protocol classes and avoid routing/protocol deadlock. It is neither the lowest-buffer nor the lowest-latency design, but it is robust.

The reason it persists is not nostalgia. Cache-coherent multicore fabrics have request, response, data, snoop/probe, and sometimes retry/ack classes. Even if the routing topology is deadlock-free, the coherence protocol can create dependency cycles: a request may require a response, a response may require data-buffer availability, and probes can block unrelated traffic. Separate virtual networks or physically separate channels break these cycles. Arm CHI, for example, exposes separate REQ/RSP/SNP/DAT channels and L-credit flow control at the link layer [25].

6.2 Adaptive routing and escape resources

Adaptive routing helps with nonuniform traffic, but it complicates deadlock freedom. Turn-model routing forbids selected turns to break cycles [26, 27]. Duato’s theory generalizes the design rule: adaptive channels can be used if there is a connected deadlock-free escape subnetwork and dependencies are constrained appropriately [28]. In a NoC this often means one deterministic escape VC plus one or more adaptive VCs.

Regional congestion awareness and DyAD-style designs show how congestion signals can guide routing [29, 30]. The caveat is verification: adaptive routing can introduce reordering, starvation, QoS interference, and corner-case protocol interactions.

6.3 Bypass: EVC and SMART

Express virtual channels and SMART both attack per-hop router latency. EVCs provide multi-hop logical channels that bypass intermediate buffering/arbitration [15]. SMART performs setup and then allows single-cycle multi-hop traversal across repeated links when resources are free [17]. These mechanisms are attractive for low-load latency and diameter-heavy topologies, but harder to time and arbitrate than conventional one-hop-per-cycle routers. They are also less helpful at saturation, where contention rather than router pipeline latency dominates.

6.4 Elastic, bufferless, and minimally buffered NoCs

Buffers consume area, leakage, and allocator complexity. Elastic-buffer flow control stores flits in pipeline registers along links, reducing router input buffering [7]. Bufferless deflection removes most buffers and always forwards flits, misrouting some under contention [8, 9]. MinBD adds a small side buffer to recover much of the performance loss while retaining energy benefits [19]. Clumsy flow control is another attempt to raise throughput in bufferless NoCs without returning to full VC buffering [31].

The mechanism-level lesson is close to HiRD: when you remove buffers, the network itself becomes the queue. This can be elegant at low to moderate load, but dynamic energy and tail latency can rise because packets travel extra hops.

7 Industry signals

Public industry documentation rarely reveals exact VC counts, credit pipelines, allocator policies, or deadlock proofs. Still, several sources are useful.

7.1 Ring-based processors

Intel Sandy Bridge-era rings. Public descriptions of Sandy Bridge report a coherent ring interconnect connecting cores, LLC slices, graphics, and the system agent, with separate request, snoop, acknowledge, and data rings [20]. The key flow-control implication is traffic-class separation: Intel exposed separate physical rings for coherence classes rather than a single undifferentiated packet stream. Exact credit and arbitration internals are not public.

IBM Cell EIB. Cell’s EIB is a stronger public flow-control example: four data rings plus a command bus, pipelined circuit-switched transport, and end-to-end command credits for outstanding requests [21]. It shows that commercial ring fabrics often prefer scheduled or credit-controlled transfer over fully general wormhole routing.

7.2 Mesh and coherent-fabric processors

Tilera TILE64 iMesh. TILE64 used five independent 2D mesh networks, each specialized for different traffic. Wentzlaff et al. describe three-entry credit-based flow control on tile-to-tile boundaries and separate physical networks for isolation, flow control, prioritization, and deadlock avoidance [32]. This is one of the clearest public processor NoC sources linking topology, credits, and protocol deadlock.

Arm CHI and CMN. Arm CHI is a coherent interconnect protocol rather than a single topology. It defines protocol channels and L-credit flow control, and requires the network layer to support deadlock-free switching [25]. Linux kernel documentation describes CMN-600 as a configurable rectangular mesh of crosspoints with CHI agents [33]. Together these are strong evidence for the industry pattern: channelized coherent traffic plus credit-controlled links over a mesh-like fabric.

Intel Xeon Scalable and Xeon Phi. Intel’s public mesh white paper describes the move from rings to a 2D mesh with distributed cores, LLC slices, CHAs, snoop filters, memory controllers, and UPI agents [34]. Knights Landing similarly used a tiled 2D mesh with distributed tag directories [35]. Public sources confirm topology and coherent-agent organization, but not router-level flow control.

8 Design implications for a ring or hierarchical-ring NoC

If the target topology resembles HiRD, the most important decisions are not just “ring or mesh,” but which congestion response is acceptable.

8.1 When a ring is a good fit

A ring or hierarchical ring is attractive when:

- most traffic is local or can be clustered naturally;
- router area and verification simplicity are more important than peak bisection bandwidth;
- the floorplan benefits from predictable unidirectional or bidirectional channels;

- coherence traffic classes can be separated by physical rings, VCs, or carefully designed virtual networks;
- worst-case all-to-all traffic is not the dominant regime.

8.2 Flow-control options for a modern hierarchical ring

1. **Conservative baseline: credit/bubble local rings plus separated traffic classes.** This is easiest to reason about. Use credits or ready/valid between ring stops, preserve bubbles on cyclic resources, and separate request/response/data/probe classes either physically or virtually. Good for correctness and bounded behavior; less elegant under long cyclic backpressure.
2. **HiRD-like deflection: buffer-light local/global rings.** Use ejection-first arbitration, bridge-transfer priority over injection, and age or injection-throttling rules to guarantee progress. This minimizes buffering and avoids global stalls. It is best when extra laps are rare; it needs careful tail-latency evaluation.
3. **Hybrid minimally buffered deflection.** Add tiny side buffers at bridge/ejection hot spots. This borrows from MinBD: a small amount of buffering can remove many harmful deflections while keeping the router simpler than a full VC design [19]. For a hierarchical ring, bridge nodes are the obvious place to spend the buffer budget.
4. **VC-enhanced ring for coherence-heavy systems.** If strict message-class isolation is non-negotiable, add VCs or physically separate rings. This sacrifices some simplicity but aligns with industrial coherent-fabric practice.

8.3 What to measure

A ring-flow-control evaluation should not stop at average latency. The key metrics are:

- zero-load latency and saturation throughput;
- 95th/99th percentile latency under bursty and adversarial traffic;
- bridge utilization and deflection rate per bridge;
- injection stall rate or injection throttle duty cycle;
- number of extra laps per delivered packet;
- energy per delivered flit, including wasted deflection traversal;
- fairness across nodes, especially nodes near/away from bridges;
- protocol deadlock freedom with request, response, data, and snoop/probe classes.

For your topology interest, the hidden boss fight is bridge congestion. A hierarchical ring can look excellent under local traffic but collapse into a global-ring/bridge bottleneck under permutation, hotspot, or coherence-snoop-heavy traffic.

9 Open problems and current research directions

- **Tail latency in bufferless rings.** Deflection papers often report average behavior and throughput, but production coherent fabrics care about outliers. Age-based priority and throttling help but can reduce injection bandwidth.

- **Protocol-aware flow control.** Many academic NoC evaluations use synthetic traffic or simplified cache-coherence models. Real request/response/probe/data dependencies can dominate correctness and sizing.
- **Hybrid buffering at structural bottlenecks.** Full buffering everywhere is expensive; no buffering anywhere is fragile. Hierarchical rings strongly suggest asymmetric buffering: spend queues at bridges, memory controllers, and ejection hotspots.
- **Physical-design co-optimization.** Rings are attractive partly because wires are predictable. SMART/EVC-like bypass and high-radix designs may win architecturally but lose timing/area battles at physical implementation.
- **QoS and isolation.** Heterogeneous multicore/accelerator SoCs need bandwidth/latency isolation. Token or slot rings have natural scheduling hooks; deflection designs need explicit admission and priority mechanisms.

10 Bottom-line recommendation

For a ring-based NoC study, I would treat the state of the art as a three-way comparison:

1. **Credit/bubble hierarchical ring:** robust baseline, easier proof, moderate buffering.
2. **HiRD-like deflection hierarchical ring:** buffer-light, simple routers, no global backpressure, but sensitive to bridge contention and tail latency.
3. **Hybrid minimally buffered deflection ring:** likely best research direction if the objective is a practical modern ring. Add small buffers only at high-leverage points and use deflection as the fallback rather than the only congestion response.

Against mesh NoCs, the ring design should be judged not only by throughput, but by router simplicity, floorplan compatibility, energy per delivered flit, and coherence verification cost. If all-to-all or many-to-memory traffic dominates, a mesh or hierarchical mesh with credit-based VC flow control is probably safer. If locality is strong and wire/router simplicity is paramount, hierarchical rings with carefully engineered deflection remain a credible design point.

References

- [1] Rachata Ausavarungnirun, Chris Fallin, Xiangyao Yu, Kevin Kai-Wei Chang, Greg Nazario, Reetuparna Das, Gabriel H. Loh, and Onur Mutlu. Design and evaluation of hierarchical rings with deflection routing. In *2014 IEEE 26th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 230–237, 2014. doi: 10.1109/SBAC-PAD.2014.31.
- [2] William J. Dally. Virtual-channel flow control. In *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*, pages 60–68, 1990. doi: 10.1145/325164.325115.
- [3] William J. Dally. Virtual-channel flow control. *IEEE Transactions on Parallel and Distributed Systems*, 3(2):194–205, 1992. doi: 10.1109/71.127260.
- [4] Parviz Kermani and Leonard Kleinrock. Virtual cut-through: A new computer communication switching technique. *Computer Networks*, 3(4):267–286, 1979. doi: 10.1016/0376-5075(79)90032-1.
- [5] Lionel M. Ni and Philip K. McKinley. A survey of wormhole routing techniques in direct networks. *Computer*, 26(2):62–76, 1993. doi: 10.1109/2.191995.

- [6] William J. Dally and Brian Towles. *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004. ISBN 978-0122007514.
- [7] George Michelogiannakis, James Balfour, and William J. Dally. Elastic-buffer flow control for on-chip networks. In *Proceedings of the 15th International Symposium on High-Performance Computer Architecture (HPCA)*, pages 151–162, 2009. doi: 10.1109/HPCA.2009.4798250.
- [8] Thomas Moscibroda and Onur Mutlu. A case for bufferless routing in on-chip networks. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*, pages 196–207, 2009. doi: 10.1145/1555754.1555781.
- [9] Chris Fallin, Chris Craik, and Onur Mutlu. Chipper: A low-complexity bufferless deflection router. In *Proceedings of the 17th International Symposium on High-Performance Computer Architecture (HPCA)*, pages 144–155, 2011. doi: 10.1109/HPCA.2011.5749724.
- [10] William J. Dally and Charles L. Seitz. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Transactions on Computers*, C-36(5):547–553, 1987. doi: 10.1109/TC.1987.1676939.
- [11] William J. Dally and Brian Towles. Route packets, not wires: On-chip interconnection networks. In *Proceedings of the 38th Design Automation Conference (DAC)*, pages 684–689, 2001. doi: 10.1145/378239.379048.
- [12] Luca Benini and Giovanni De Micheli. Networks on chips: A new soc paradigm. *Computer*, 35(1): 70–78, 2002. doi: 10.1109/2.976921.
- [13] Li-Shiuan Peh and William J. Dally. A delay model and speculative architecture for pipelined routers. In *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, pages 255–266, 2001. doi: 10.1109/HPCA.2001.903268.
- [14] Robert Mullins, Andrew West, and Simon Moore. Low-latency virtual-channel routers for on-chip networks. In *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*, pages 188–197, 2004. doi: 10.1109/ISCA.2004.1310774.
- [15] Amit Kumar, Li-Shiuan Peh, Partha Kundu, and Niraj K. Jha. Express virtual channels: Towards the ideal interconnection fabric. In *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, pages 150–161, 2007. doi: 10.1145/1250662.1250681.
- [16] John Kim, William J. Dally, and Dennis Abts. Flattened butterfly: A cost-efficient topology for high-radix networks. In *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, pages 126–137, 2007. doi: 10.1145/1250662.1250679.
- [17] Tushar Krishna, Chia-Hsin Owen Chen, Woo Cheol Kwon, and Li-Shiuan Peh. Breaking the on-chip latency barrier using smart. In *Proceedings of the 19th International Symposium on High Performance Computer Architecture (HPCA)*, pages 378–389, 2013. doi: 10.1109/HPCA.2013.6522334.
- [18] George Michelogiannakis, Daniel Sanchez, William J. Dally, and Christos Kozyrakis. Evaluating bufferless flow control for on-chip networks. In *Proceedings of the Fourth ACM/IEEE International Symposium on Networks-on-Chip*, pages 9–16, 2010. doi: 10.1109/NOCS.2010.10.
- [19] Chris Fallin, Greg Nazario, Xiangyao Yu, Kevin Chang, Rachata Ausavarungnirun, and Onur Mutlu. Minbd: Minimally-buffered deflection routing for energy-efficient interconnect. In *Proceedings of the Sixth ACM/IEEE International Symposium on Networks-on-Chip*, pages 1–10, 2012. doi: 10.1109/NOCS.2012.8.

- [20] David Kanter. Intel’s sandy bridge microarchitecture. Real World Technologies, 2010. URL <https://www.realworldtech.com/sandy-bridge/8/>.
- [21] Thomas W. Ainsworth and Timothy M. Pinkston. On characterizing performance of the cell broadband engine element interconnect bus. Technical report, University of Southern California, CENG Technical Report, 2006. URL <https://ceng.usc.edu/techreports/2006/Pinkston%20CENG-2006-15.pdf>.
- [22] K. Ravindran, R. S. Iyer, and L. Bhuyan. On topology and bisection bandwidth for hierarchical-ring networks for shared memory multiprocessors. In *Proceedings of the Fourth International Symposium on High-Performance Computer Architecture (HPCA)*, 1998.
- [23] Reetuparna Das, Soumya Eachempati, Abhishek K. Mishra, Vijaykrishnan Narayanan, and Chita R. Das. Design and evaluation of hierarchical on-chip network topologies for next generation cmps. In *Proceedings of the 15th International Symposium on High-Performance Computer Architecture (HPCA)*, 2009. doi: 10.1109/HPCA.2009.4798255.
- [24] C. Carrion, R. Beivide, J. A. Gregorio, and F. Vallejo. A flow control mechanism to avoid message deadlock in k-ary n-cube networks. In *Proceedings of the Fourth International Conference on High-Performance Computing*, pages 322–329, 1997. doi: 10.1109/HIPC.1997.634510.
- [25] Arm Ltd. Amba 5 chi architecture specification, ihi 0050e.b, 2021. URL <https://documentation-service.arm.com/static/6295e03db334256d9ea8ae86>.
- [26] Christopher J. Glass and Lionel M. Ni. The turn model for adaptive routing. *Journal of the ACM*, 41(5):874–902, 1994. doi: 10.1145/185675.185682.
- [27] Ge-Ming Chiu. The odd-even turn model for adaptive routing. *IEEE Transactions on Parallel and Distributed Systems*, 11(7):729–738, 2000. doi: 10.1109/71.877831.
- [28] Jose Duato. A new theory of deadlock-free adaptive routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems*, 4(12):1320–1331, 1993. doi: 10.1109/71.250114.
- [29] Jingcao Hu and Radu Marculescu. Dyad: Smart routing for networks-on-chip. In *Proceedings of the 41st Annual Design Automation Conference (DAC)*, 2004. doi: 10.1145/996566.996638.
- [30] Paul Gratz, Boris Grot, and Stephen W. Keckler. Regional congestion awareness for load balance in networks-on-chip. In *Proceedings of the 14th International Symposium on High-Performance Computer Architecture (HPCA)*, 2008. doi: 10.1109/HPCA.2008.4658640.
- [31] Hanjoon Kim, Yonggon Kim, and John Kim. Clumsy flow control for high-throughput bufferless on-chip networks. *IEEE Computer Architecture Letters*, 12(2):47–50, 2013. doi: 10.1109/L-CA.2012.22.
- [32] David Wentzlaff, Patrick Griffin, Henry Hoffmann, Liewei Bao, Bruce Edwards, Carl Ramey, Matthew Mattina, Chyi-Chang Miao, John F. Brown III, and Anant Agarwal. On-chip interconnection architecture of the tile processor. *IEEE Micro*, 27(5):15–31, 2007. doi: 10.1109/MM.2007.89.
- [33] Linux Kernel Documentation. Arm coherent mesh network pmu, 2023. URL <https://www.kernel.org/doc/html/v6.3/admin-guide/perf/arm-cmn.html>.
- [34] Akhilesh Kumar. New intel mesh architecture: The superhighway of the data center. Intel White Paper, 2017. URL https://silix.com.br/pdf/Intel/Intel_Mesh_Whitepaper.pdf.
- [35] Avinash Sodani. Knights landing (knl): 2nd generation intel xeon phi processor. Hot Chips 27, 2015. URL <https://aiichironakano.github.io/cs653/Sodani-KnightsLanding-Hotchips15.pdf>.